DiScammer — обзор проекта

Что это

DiScammer — образовательная платформа по кибербезопасности и распознаванию онлайн-мошенничества. Сайт публикует гайды по типам скама, интервью с экспертами, рассылки, статьи и обучающий курс; отдельной фичей встроена интерактивная Unity WebGL-игра «Cyber Watch» (`/play/`).

Контент — на английском, статичный, переезжает с исходного Tilda-сайта по плану в DISCAMMER_REDESIGN_PLAN_v2.md (русский).

Текущий объём контента:

- 10 гайдов (`content/guides/`) — типы скама с уровнем угрозы.
- 8 интервью (`content/interviews/`).
- 8 рассылок (`content/newsletters/`).
- 7 статей (`content/articles/`).
- Курс из модулей, захардкоженных в `backend/seed_data.py` (`COURSE_MODULES`).

Прод-домен: **discammer.com**.

Архитектура

Сервер-рендеринг на FastAPI + Jinja2 с **islands-гидрацией** на React: Jinja отдаёт полноценный HTML, а React монтируется только в помеченные `[data-component]` элементы.

Поток запроса

1. `backend/main.py` поднимает `FastAPI` , монтирует `/static` , цепляет Jinja2 к `app.state.templates` и подключает три роутера.
2. `routers/pages.py` рендерит SSR-страницы: `/` , `/journal` , `/journal/{slug}` , `/guide/{slug}` , `/course` , `/game` . Markdown из БД конвертируется в HTML на сервере через `markdown_renderer.render_markdown` и инжектится в шаблон.
3. `routers/api.py` отдаёт JSON под `/api/posts` , `/api/guides` , `/api/course/modules` — для тех островов, что подгружают данные на клиенте.
4. `routers/game.py` обслуживает Unity WebGL-сборку под `/play/` , корректно проставляя `Content-Encoding: br` для предсжатых `.wasm.br` / `.data.br` / `.framework.js.br` .
5. Все страницы наследуют `templates/base.html` , который тянет `/static/css/main.css` и `/static/js/app.js` .

Острова (islands)

`frontend/src/main.tsx` — единственная точка входа, бандлится в `backend/static/js/app.js`. По `DOMContentLoaded` он сканирует DOM на `[data-component]`, ищет имя в lazy-реестре `components`, парсит `data-props` (JSON) и монтирует React-компонент в этот элемент через `createRoot`. SSR-разметка сохраняется как `dangerouslySetInnerHTML` — на случай, если гидрация не сработала.

Зарегистрировано **12 React-островов** в `frontend/src/components/`:

`HeroTypewriter`, `HeroTerminal`, `ParticleBackground`, `ScanlineOverlay`, `ContentCarousel`, `ScamTypesGrid`, `GlitchText`, `TerminalBlock`, `JournalFilter`, `CourseProgress`, `GameLauncher`, `MobileMenu`.

Добавление нового виджета — три изменения: компонент `.tsx` в `frontend/src/components/` → регистрация в `components` в `main.tsx` → `<div data-component="Name" data-props='...'></div>` в Jinja-шаблоне → ребилд фронта.

Технический стек

Backend — Python

Компонент	Версия	Назначение
Python	3.12 (прод-контейнер) / 3.14 (локально)	Рантайм
FastAPI	≥ 0.115	Веб-фреймворк
Uvicorn	≥ 0.34 (`[standard]`)	ASGI-сервер
Jinja2	≥ 3.1	Шаблонизатор для SSR
aiosqlite	≥ 0.20	Асинхронный драйвер SQLite
markdown	≥ 3.7	Парсер Markdown → HTML
bleach	≥ 6.0	Санитизация HTML
python-multipart	≥ 0.0.18	Парсинг multipart-форм
aiofiles	≥ 24.1	Асинхронный I/O

Зависимости — в requirements.txt; dev-зависимости (pytest) — в requirements-dev.txt.

Frontend — TypeScript / React

Компонент	Версия	Назначение
React + ReactDOM	18.3	UI-острова
TypeScript	5.7	Типизация
Vite	6.0	Сборщик и dev-сервер
`@vitejs/plugin-react`	4.3	React + Fast Refresh
TailwindCSS	4.0	Стили (через `@tailwindcss/vite`)
`@types/react`, `@types/react-dom`	18.3	Типы

Зависимости — в frontend/package.json.

Хранилище и контент

- **SQLite** через `aiosqlite`, файл — `data/discammer.db` (в `.gitignore`, монтируется как volume в docker-compose).
- Три таблицы определены инлайн в `backend/database.py`: `post` (interview/newsletter/article различаются по полю `type`), `scam_guide`, `course_module`.
- `init_db()` создаёт таблицы; `seed_database()` вызывается через FastAPI lifespan-хук и **на каждом старте чистит + переливает** `post` / `scam_guide` / `course_module`. Безопасно: эти таблицы хранят только производный из markdown контент, пользовательских данных там нет.

Контент-пайплайн

Источник правды — markdown-файлы в `content/{guides,interviews,newsletters,articles}/`. `backend/seed_data.py` сам парсит YAML-подобный frontmatter регуляркой (не PyYAML — поддерживаются только плоские `key: value` пары и приведение к int; никаких вложенных структур, никаких списков).

Поля frontmatter, которые читает сидер: `slug`, `title`, `subtitle` / `description`, `type`, `icon`, `threat_level`, `category`, `cover_image`, `published_at`. Для interview/newsletter/article `type` по умолчанию = имя поддиректории без хвостовой `s`.

Модули курса — захардкожены в `COURSE_MODULES` внутри `seed_data.py`, не подтягиваются из файлов.

Стилизация

TailwindCSS v4 через плагин `@tailwindcss/vite`. Дизайн-токены — в `frontend/src/styles/globals.css` внутри `@theme`. Палитра — киберпанк:

Токен	Значение
Фон	`#0B0E17`
Cyan-акцент	`#00F0FF`
Green	`#39FF6B`
Red	`#FF4757`
Amber	`#FFB547`

Директива `@source "../../../backend/templates/**/*.html"` в `globals.css` говорит Tailwind v4 сканировать Jinja-шаблоны — иначе классы, используемые только в `.html`, выпиливаются purge-стадией.

Игра

Pre-built Unity WebGL-бандл лежит в `backend/game_build/` (в репо). Раздаётся отдельным роутером `routers/game.py`:

- Файлы под `Build/` (версионированные `web_gl_build_10.*`) — `Cache-Control: public, max-age=31536000, immutable`.
- Прочие — `max-age=86400`.
- `*.br` отдаются с `Content-Encoding: br` и корректным inner `Content-Type` (`application/wasm`, `application/javascript`, `application/octet-stream`).
- `/play` → 307 редирект на `/play/`, чтобы относительные пути в `index.html` резолвились.

Структура репозитория

```
DiScammer/
├── backend/
│   ├── main.py                  # FastAPI app + lifespan + Jinja2 env
│   ├── config.py                # пути (STATIC_DIR, TEMPLATES_DIR, DATABASE_PATH...)
│   ├── database.py              # SQL DDL + CRUD-хелперы (aiosqlite)
│   ├── models.py                # Pydantic-модели для API-ответов
│   ├── seed_data.py             # парсер frontmatter + COURSE_MODULES
│   ├── markdown_renderer.py     # markdown → bleach-sanitized HTML
│   ├── routers/
│   │   ├── pages.py             # SSR-страницы
│   │   ├── api.py               # /api/* JSON
│   │   └── game.py              # /play/* Unity WebGL
│   ├── templates/               # Jinja2: base, home, journal, post, guide, course, game
│   ├── static/                  # собранные css/main.css и js/app.js (артефакт Vite)
│   ├── game_build/              # Unity WebGL (предсжатые .br)
│   └── tests/
├── frontend/
│   ├── package.json
│   ├── vite.config.ts
│   ├── tsconfig.json
│   └── src/
│       ├── main.tsx             # entry — гидрация островов
│       ├── components/          # 12 React-островов
│       ├── styles/globals.css   # @theme tokens + @source для Jinja
│       └── utils/
├── content/
│   ├── guides/                  # 10 файлов
│   ├── interviews/              # 8 файлов
│   ├── newsletters/             # 8 файлов
│   └── articles/                # 7 файлов
├── data/                        # SQLite (gitignored, persistent volume)
├── scripts/
│   └── deploy.sh                # вызывается по SSH с GitHub Actions
├── .github/workflows/ci-cd.yml  # tests → build → docker build → SSH deploy
├── Dockerfile                   # multi-stage: node:20 → python:3.12
├── docker-compose.yml           # базовый
├── docker-compose.prod.yml      # прод-оверрайды
├── requirements.txt
├── requirements-dev.txt
├── CLAUDE.md                    # инструкции для агентов
└── DISCAMMER_REDESIGN_PLAN_v2.md  # дизайн-спека (русская)
```

Сборка и разработка

Backend (из `backend/`)

```
.venv/bin/uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Важно: `main:app` нужно поднимать **из директории** `backend/` — `main.py` подкидывает её в `sys.path`, иначе sibling-импорты вида `from database import ...` ломаются. `uvicorn backend.main:app` из корня — не работает.

Установка зависимостей (из корня):

```
pip install -r requirements.txt
```

Frontend (из `frontend/`)

```
npm install
npm run dev        # Vite dev-сервер (для итераций по компонентам)
npm run build      # tsc типчек + Vite build → ../backend/static/
npm run preview    # превью бандла
```

`outDir` Vite — `../backend/static` с `emptyOutDir: false`. Сборка **перезаписывает** `backend/static/js/app.js` и `backend/static/css/main.css` на месте; отдельного `frontend/dist/` нет. После правки любого React-компонента нужен ребилд, чтобы FastAPI отдавал свежий бандл.

`rollupOptions.output.inlineDynamicImports: true` — все lazy-чанки React.lazy инлайнятся в `app.js`. Это сознательно: без инлайнинга кэш-бастинг через `?v=...` приводил к двойной инстанциации React и ошибке #321 (Invalid hook call).

Полный стек локально

```
docker compose up --build    # билд фронта → бэкенд-имидж, слушает :8000
```

Тесты

В корне есть `pytest.ini`, в `backend/tests/` — pytest-тесты. CI прогоняет `pytest -q`. (Файлы `test_*.py` под `backend/.venv/` — это библиотечные, не наши.)

CI/CD

`.github/workflows/ci-cd.yml` — четыре джобы:

1. **backend-tests** — Python 3.12, `pytest -q`.
2. **frontend-build** — Node 20, `npm ci` + `npm run build` (tsc типчек включён в build-скрипт).
3. **docker-build** — `docker buildx`, билд без push, с GHA-кэшем.
4. **deploy** — только на пуш в `main`. Подключается по SSH (`appleboy/ssh-action`), вызывает `bash /opt/discammer/scripts/deploy.sh` на сервере. После — health-check: ждёт `GET /` → 200 (до 10 попыток с шагом 5 сек), затем прогоняет `/`, `/journal`, `/course`, `/game`, `/api/posts`, `/api/guides`.

Сервер: `/opt/discammer` — клон репо. `scripts/deploy.sh` делает `git fetch && git reset --hard origin/main`, поднимает `docker compose -f docker-compose.yml -f docker-compose.prod.yml up -d --build` и подчищает dangling-имиджи старше 24 часов (один свежий оставляется как rollback-кандидат).

GH-секреты для деплоя: `DEPLOY_HOST`, `DEPLOY_USER`, `DEPLOY_SSH_KEY`.

Docker

Multi-stage Dockerfile:

1. **Стадия `frontend-build`** (`node:20-slim`) — копирует `frontend/`, существующий `backend/static/` (на случай партиал-апдейтов) и `backend/templates/` (Tailwind v4 их сканирует через `@source`, без них классы вычищаются), запускает `npm ci && npm run build`.
2. **Стадия рантайма** (`python:3.12-slim`) — ставит `requirements.txt`, копирует `backend/`, `content/`, готовый `backend/static/` из первой стадии. `EXPOSE 8000`, `WORKDIR /app/backend`, `CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]`.

`docker-compose.yml` — сервис `web`, проброс `:8000`, volume `./data:/app/data` (БД переживает рестарт), `restart: unless-stopped`.

Эндпоинты

Страницы (HTML, SSR)

Маршрут	Шаблон	Что делает
`GET /`	`home.html`	Главная: 4 свежих interview/newsletter/article + все гайды + статы
`GET /journal`	`journal.html`	Лента всех постов + гайдов
`GET /journal/{slug}`	`post.html`	Детальная страница поста + 3 related того же типа
`GET /guide/{slug}`	`guide.html`	Детальная страница гайда + 9 related
`GET /course`	`course.html`	Список модулей курса
`GET /game`	`game.html`	Лендинг с лаунчером Unity-игры

JSON API

Маршрут	Ответ
`GET /api/posts?type=&limit=`	`List[PostResponse]`
`GET /api/guides`	`List[ScamGuideResponse]`
`GET /api/course/modules`	`List[CourseModuleResponse]`

Игра

Маршрут	Что отдаёт
`GET /play`	307 → `/play/`
`GET /play/`	`game_build/index.html` (no-cache)
`GET /play/{path}`	Любой ассет из `game_build/` с корректным MIME, brotli-заголовками и cache policy

Статика

`GET /static/*` — Vite-артефакты (`css/main.css` , `js/app.js` и пр.). В шаблонах URL'ы оборачивает helper `asset_url(path)` из `main.py` — добавляет `?v=<mtime>` к финальному URL, чтобы браузер не залипал на старом бандле после деплоя (Vite даёт стабильные имена без хеша).

Известные особенности

- **Python 3.14 + Jinja2 `LRUCache` несовместимы локально.** В `main.py` `Environment` собирается вручную с `cache_size=0` и передаётся в `Jinja2Templates(env=...)` . Не «упрощать» обратно в `Jinja2Templates(directory=...)` — сломается локалка. В прод-контейнере (3.12) безвредно.
- **React дедуп.** В `vite.config.ts` стоит `resolve.dedupe: ['react', 'react-dom', 'react/jsx-runtime']` — иначе lazy-чанки тянут второй экземпляр React, и хуки ломаются (Invalid hook call, error #321).
- **Сидер пересоздаёт контент-таблицы на каждом старте.** Это by design — markdown — источник правды. Любая правка frontmatter или тела применится после следующего рестарта контейнера.
- **`backend/static/` коммитится в репо** (Vite-артефакт), потому что Dockerfile частично переиспользует существующий каталог. Не забывать ребилдить фронт и коммитить вместе с правками компонентов.

Связанные документы

- CLAUDE.md — инструкции для AI-агентов, краткая выжимка из этого документа.
- DISCAMMER_REDESIGN_PLAN_v2.md — авторитетная спека редизайна и миграции контента с Tilda (русский).